Optimal Ship Routing System

Project demonstration guide

A weather-aware routing prototype for vessels travelling around India, combining safe-water constraints, wave data, pathfinding, and an ML-based speed model.

Recommended demo: Cochin (Kerala) to Dubai / Jebel Ali (UAE)

1. The project in one minute

Nautilus answers a practical question: should a ship follow the shortest safe route, or take a slightly longer route to avoid difficult sea conditions? The system models the sea around India as a navigable grid and compares two searches on the same grid.

Component	Role in the system
GEBCO bathymetry	Identifies water depth and removes shallow cells.
ERA5 wave data	Provides significant wave height, direction, and period.
Safety mask	Keeps routes in open water and away from the coast buffer.
Dijkstra	Traditional baseline: minimum safe distance.
A* + ML	Proposed method: travel-time cost adjusted for vessel speed loss in waves.
Web interfaces	Flask + Google Maps for the polished demo; Streamlit for an offline fallback.

Core message for the audience

The proposed route is not simply the shortest line on a map. It is a safe route whose cost reflects how waves and vessel characteristics affect achievable speed. The result is a transparent trade-off between distance, time, and sea state.

2. What happens inside

The system has an offline preparation stage and a fast demo stage.

Stage	What happens	Output
Preprocess	Extract the configured region from large NetCDF files, downsample, and aggregate wave scenarios.	bathymetry.npz, weather.npz, weather_grid.csv
Build grid	Mark cells navigable only when depth and coast-distance rules are satisfied.	grid.npz
Train/load model	Random Forest predicts attainable speed from waves and vessel parameters.	speed_model.joblib
Route request	Snap ports to grid cells and run Dijkstra and/or A*.	Path, distance, time, waves, explored cells

Important configuration

- Region: latitude 0 to 27 N, longitude 50 to 100 E.
- Grid: 0.05 degrees, approximately 5.5 km per cell.
- Safety: minimum depth 10 m and coast buffer 22 km.
- A* weather cost uses wave height, head-sea direction, wave period, and the ML-predicted speed multiplier.

3. Prepare the demo

The repository already contains the processed artifacts needed for normal startup. For a fresh machine, install Python 3.10 or newer and the dependencies first.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Choose one interface

Interface	Command	When to use
Flask + Google Maps	$env:GOOGLE_MAPS_API_KEY = "your-key" python server.py	Best visual presentation. Requires a valid Google Maps JavaScript API key.
Streamlit + Folium	streamlit run app.py	Best fallback when no Google Maps key is available. Works from local processed data.

Open the application at http://localhost:8000 for Flask or http://localhost:8501 for Streamlit.

If the processed artifacts are missing

```
python -m src.preprocess
python -m src.ml_model
```

Preprocessing reads the large raw NetCDF files and can require several GB of memory. It is normally a one-time operation; the demo itself loads the smaller files in data/processed/.

NetCDF (Network Common Data Form)

4. Recommended live demonstration

Step	Action	What to say / show
1	Introduce the problem	Explain that a shortest-distance route can cross rougher water, slow the vessel, and increase operational risk.
2	Set the voyage	Select Container Ship, Cochin (Kerala) as Origin, and Dubai / Jebel Ali (UAE) as Destination.
3	Show the baseline	Keep Dijkstra and Weather-aware A* enabled. Keep the Wave heatmap and ML speed model enabled.
4	Run typical weather	Click Find optimal route. Point out the blue traditional route, red proposed route, endpoint markers, wave colors, and comparison cards.
5	Switch to rough weather	Select Rough and run again. Explain that the same voyage is re-evaluated under a more difficult sea state.
6	Inspect the evidence	Compare distance, estimated time, maximum wave, minimum coast distance, and cells explored. Click a route waypoint to show local wave height, period, and direction.
7	Close with the trade-off	A* may add distance when that reduces peak waves. The correct route depends on vessel, weather scenario, and the chosen operational objective.

Visual key

Visual	Meaning
Blue line	Dijkstra: traditional shortest safe-distance route.
Red line	Weather-aware A*: proposed route, with animated direction marker.

Visual	Meaning
Wave overlay	Color-coded significant wave height across the region.
Waypoint marker	Click to inspect wave height, wave period, and direction.

5. How to explain the algorithms

Dijkstra explores the 8-connected navigable grid and adds only the geographic distance of each move. It is the control route: safe according to the bathymetry/coast mask, but indifferent to weather.

A* uses the same safe grid, but adds a weather multiplier to each move and uses a distance heuristic to focus the search. With the ML model enabled, the multiplier is based on service speed divided by predicted attainable speed. A head sea, larger waves, shorter period, or a more wave-sensitive vessel increases the effective cost.

The implementation reports estimated time using the route distance and the configured service speed. The weather-aware search cost is what changes the path; the result cards additionally report the actual wave statistics sampled along each route.

6. Results already produced by the project

The included evaluation.csv compares traditional and proposed routes in rough-weather evaluation runs. These values are useful as a prepared backup when a live run is slow or unavailable.

Voyage	Distance km D / A*	Peak wave m D / A*	Cells explored D / A*
Mumbai -> Chennai	2953.6 / 2954.9	3.17 / 3.17	249,000 / 55,321
Kandla -> Cochin	1780.2 / 1780.3	3.38 / 3.38	97,263 / 12,497
Cochin -> Dubai	2771.2 / 3068.9	4.13 / 3.31	313,745 / 83,978
Visakhapatnam -> Colombo	1697.0 / 1701.4	2.44 / 2.44	86,879 / 26,616
Chennai -> Port Blair	1612.2 / 1621.7	3.00 / 3.03	100,360 / 40,811

Best interpretation of the Cochin-Dubai row

• A* adds 297.7 km, but reduces peak wave height by 0.82 m.

• The proposed search explores about 73% fewer cells than Dijkstra in this evaluation.

• This is a decision-support demonstration, not a live navigation certification.

7. Viva questions and concise answers

Question	Answer
Why use bathymetry?	To prevent the route from crossing shallow water that may be unsafe for the vessel draft.
Why compare Dijkstra and A*?	Dijkstra provides a transparent traditional baseline; A* shows the proposed weather-aware optimization.
What does the ML model predict?	Attainable vessel speed in a given wave state, considering wave height, relative direction, period, service speed, and wave sensitivity.
Is the training data real AIS data?	No. The current model learns from a physics-inspired synthetic dataset because per-ship speed-loss logs are not public in this project.
Why can A* be longer?	It minimizes weather-adjusted travel cost, so avoiding rough water can justify extra distance.
What is live weather?	An optional weather scenario fetched through the live-weather module; typical and rough scenarios are available offline.
What are the limitations?	The region and grid resolution are fixed by configuration, weather is scenario-based, and the model should be retrained with real AIS data for production use.

8. Troubleshooting during the demo

• Blank Flask map: check GOOGLE_MAPS_API_KEY, API restrictions, billing, and the localhost origin restriction.

• Grid/model load error: run python run_all.py or the preprocessing and ML commands above.

• Live weather error: switch back to Typical or Rough; the offline scenarios do not depend on an external weather request.

• No route found: choose different ports or confirm that the selected endpoints snap to navigable cells.

One-sentence closing

Nautilus demonstrates how geospatial safety constraints, numerical pathfinding, marine weather, and vessel-specific machine learning can be combined into an explainable route recommendation.